

## PRÁCTICAS PROFESIONALIZANTES II

### Lineamientos para la entrega y autoevaluación del proyecto

Este documento define los criterios mínimos que debe cumplir un Producto Mínimo Viable (MVP) para ser aprobado en la materia Prácticas Profesionalizantes II. Está pensado como una guía de autoevaluación: antes de enviar su proyecto, cada grupo debe recorrer este checklist y verificar que los ítems obligatorios están cubiertos.

**El peso de la evaluación está en el código funcionando y la documentación es un respaldo que demuestra el proceso, no un fin en sí mismo.**

### ¿Qué se entiende por MVP en esta materia?

Un MVP es la versión más pequeña de un software real que resuelve el problema central identificado al inicio del proyecto. No es un prototipo de papel ni una demo con datos falsos: es un sistema que corre, que persiste datos y que un usuario real podría usar (aunque sea de forma limitada).

| Criterio                  | Tipo               | Descripción / Evidencia esperada                                                                                                               |
|---------------------------|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Problema real             | <b>OBLIGATORIO</b> | El sistema ataca una problemática identificada y documentada. El alcance del MVP es coherente con ese problema.                                |
| Flujo principal funcional | <b>OBLIGATORIO</b> | Al menos el flujo central de la aplicación funciona de punta a punta: el usuario puede completar la acción principal para la que fue diseñada. |
| Datos persistidos         | <b>OBLIGATORIO</b> | Los datos se guardan y recuperan correctamente (SQL Server, Firebase, o el motor de base de datos elegido).                                    |

## Gestión del proyecto

La materia requiere que el trabajo se gestione con herramientas profesionales que evidencien el proceso y la colaboración del equipo.

### Repositorio Git

| Criterio              | Tipo               | Descripción / Evidencia esperada                                                                                                                |
|-----------------------|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Repositorio existente | <b>OBLIGATORIO</b> | El proyecto tiene un repositorio en GitHub, GitLab o equivalente.                                                                               |
| Historial de commits  | <b>OBLIGATORIO</b> | Se evidencia trabajo incremental: hay commits de múltiples fechas con mensajes descriptivos. Un solo commit con todo el código no es aceptable. |
| Ramas / branches      | <b>RECOMENDADO</b> | Se usaron branches para separar funcionalidades o integrantes (feature branches, rama develop, etc.).                                           |
| README completo       | <b>OBLIGATORIO</b> | El README explica: qué hace el sistema, cómo instalarlo y correrlo localmente, tecnologías usadas, y los integrantes del grupo.                 |
| .gitignore            | <b>RECOMENDADO</b> | El repositorio incluye .gitignore apropiado para el stack (sin node_modules, bin, obj, secretos, etc.).                                         |

### Tablero Trello

| Criterio                        | Tipo               | Descripción / Evidencia esperada                                                                       |
|---------------------------------|--------------------|--------------------------------------------------------------------------------------------------------|
| Tablero existente               | <b>OBLIGATORIO</b> | El proyecto tiene un tablero Trello (u otra herramienta Kanban equivalente) compartido con el docente. |
| Columnas básicas                | <b>OBLIGATORIO</b> | El tablero evidencia organización mínima: columnas To Do / Doing / Done.                               |
| Tareas de todos los integrantes | <b>OBLIGATORIO</b> | Las tarjetas reflejan trabajo de todo el equipo, no solo de una persona.                               |
| Estado actualizado              | <b>OBLIGATORIO</b> | El tablero refleja el estado real del proyecto al momento de la entrega.                               |
| Descripción en tarjetas         | <b>RECOMENDADO</b> | Las tarjetas incluyen descripción, criterios de aceptación o checklist propios.                        |

## Código y arquitectura

No se evalúa tecnología específica: React, .NET, Angular, Ionic, PHP o cualquier otro stack acordado es válido. Lo que sí se evalúa es que el código sea organizado, funcional y razonable para el alcance del proyecto.

| Criterio                                       | Tipo               | Descripción / Evidencia esperada                                                                                                                                                       |
|------------------------------------------------|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Estructura de carpetas                         | <b>OBLIGATORIO</b> | El proyecto tiene una estructura de carpetas coherente con el stack elegido.                                                                                                           |
| Separación de responsabilidades                | <b>OBLIGATORIO</b> | Hay separación mínima entre UI, lógica de negocio y acceso a datos (por ejemplo: componentes / servicios / repositorios, o vistas / controladores / modelos).                          |
| Conexión a base de datos                       | <b>OBLIGATORIO</b> | La aplicación se conecta a una base de datos real y persiste datos. Intentar que las credenciales no están hardcodeadas en el código (usar .env o equivalente) en caso de ser posible. |
| Variables y funciones con nombres descriptivos | <b>RECOMENDADO</b> | El código es legible: los nombres de variables y funciones expresan su propósito.                                                                                                      |

## Testing mínimo

El testing no necesita ser exhaustivo, pero sí tiene que existir y documentar casos reales del sistema.

| Criterio                      | Tipo                            | Descripción / Evidencia esperada                                                                                                                                     |
|-------------------------------|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Evidencia de pruebas manuales | <b>OBLIGATORIO</b>              | Detallar un documento de pruebas adjunto que incluya un listado de casos de prueba ejecutados manualmente (funcionalidad + resultado esperado + resultado obtenido). |
| Tests unitarios               | <b>OPCIONAL – NO MANDATARIO</b> | Definir tests unitarios que cubren al menos las funciones o métodos más críticos del negocio (mínimo 3 tests relevantes).                                            |

## Presentación y defensa

El proyecto debe poder ser evaluado sin ambigüedad. El docente debe poder ver el sistema funcionando.

| Criterio                                      | Tipo               | Descripción / Evidencia esperada                                                                                                         |
|-----------------------------------------------|--------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Demo funcional                                | <b>OBLIGATORIO</b> | El sistema puede ser demostrado en vivo durante la presentación. Puede ser local.                                                        |
| Instrucciones claras para levantar el entorno | <b>OBLIGATORIO</b> | El README debe explicar cómo correrlo localmente para poder continuar con el desarrollo o hacer nuevas pruebas.                          |
| Presentación del problema/solución            | <b>OBLIGATORIO</b> | El grupo puede explicar en menos de 5 minutos: cuál es el problema, a quién afecta, y cómo el MVP lo resuelve (aunque sea parcialmente). |
| Link al repo y Trello en la entrega           | <b>OBLIGATORIO</b> | El formulario/plataforma de entrega incluye el link al repositorio y al tablero Trello.                                                  |

## Recomendaciones de seguridad básica

| Criterio                      | Tipo               | Descripción / Evidencia esperada                                                                                                |
|-------------------------------|--------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Sin credenciales en el código | <b>RECOMENDADO</b> | No hay passwords, API keys ni cadenas de conexión hardcodeadas en archivos que estén en el repositorio. Usar .env y .gitignore. |
| Validación de inputs          | <b>RECOMENDADO</b> | Los formularios y endpoints validan los datos de entrada (tipos, campos obligatorios, formatos básicos).                        |
| Autenticación si aplica       | <b>OPCIONAL</b>    | Si el sistema tiene usuarios, hay algún mecanismo de autenticación implementado (JWT, session, Firebase Auth, etc.).            |